EMBEDDED SYSTEM DESIGN EXPERIMENT

嵌入式系统设计实验

曾毓

实验 03 RTOS基本应用实验

杭州電子科技大学
HANGZHOU DIANZI UNIVERSITY

裸机系统程序结构缺陷

后台程序在main主程序中循环顺序执行，其实时性难以得到保证。前台程序用于响应异步事件，执行较为简短的操作，如果需要进行复杂的事件处理（算法或者延时等操作），则需将这些复杂操作放入后台程序中处理。

实验2的数码管显示键码程序中，如果按键扫描消抖延时过大，同一个循环中的数码管显示功能就会受到很大影响。

裸机程序要实现这样的功能要求只能用中断或者时间片计数方式，可是这样会让程序结构变得复杂而难以设计。因为中断方式虽然也可以实现打断和跳转功能，但是当程序复杂之后，过多的中断将使得裸机程序难以阅读和维护。

因此有经验的开发人员在编写裸机程序的时候都会尽力少用中断，慎用中断。

多任务操作系统

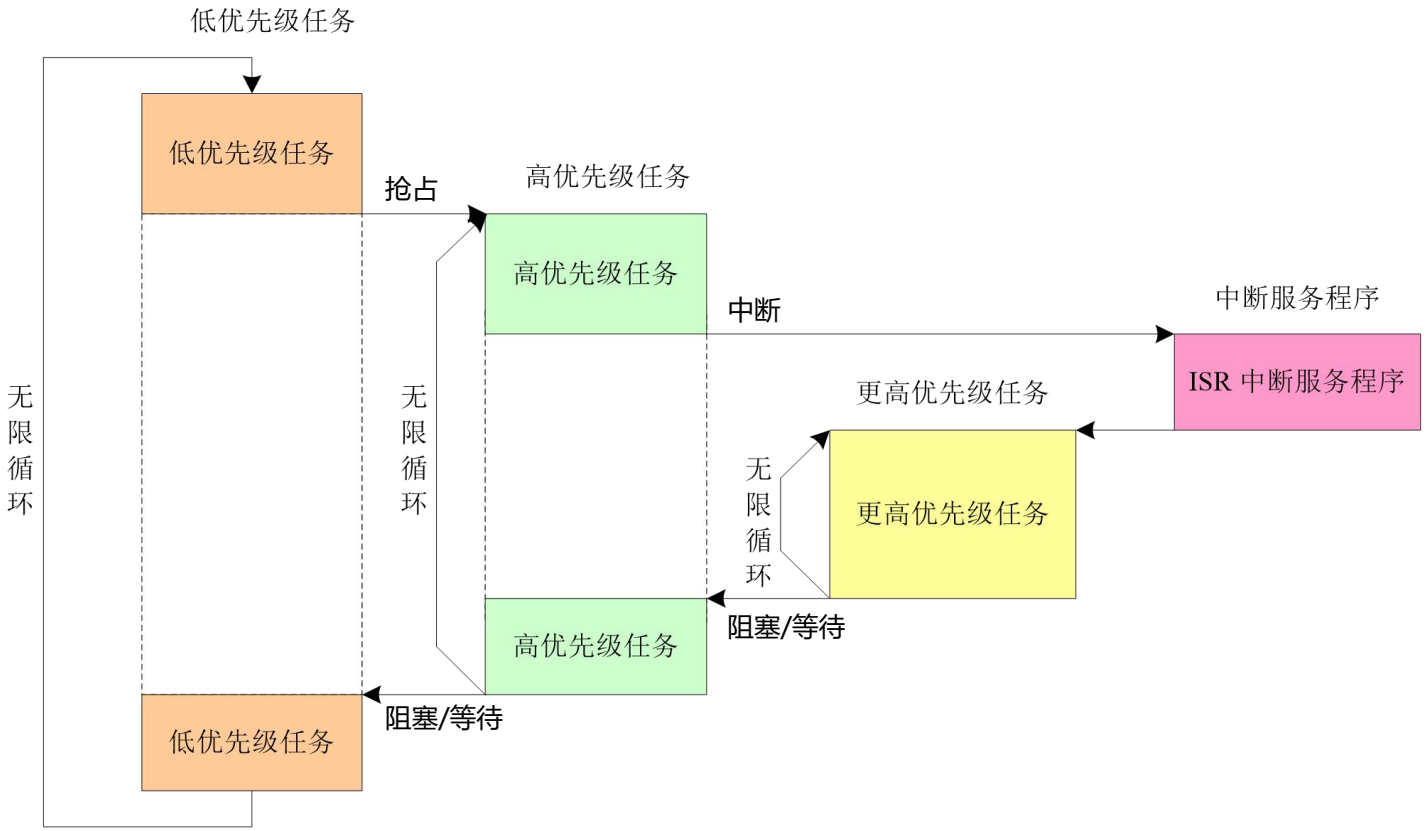
随着项目规模和逻辑复杂程度提升，仅仅依靠前后台系统来达到高实时响应的难度会越来越大。基于该原因，单片机程序设计引入多任务系统将事件处理操作放到任务中完成。因为任务与中断一样也具有优先级，所以优先级高的任务会被优先执行，多任务系统的实时性相对前后台系统就有了提高。

对多任务操作系统而言，由于每个任务都是独立互不干扰且由操作系统调度管理，编程时可以不需要精心设计程序的执行流，各个功能模块之间的干扰也大为减少，因此基于多任务系统的应用开发难度也相对减小了不少。

在嵌入式系统上可以运行的操作系统一般分两种。一种是需要有MMU硬件支持的**复杂操作系统**，比如Linux、安卓、iOS；另一种是不需要MMU的**实时操作系统**。实时操作系统即Real Time Operating System，缩写为RTOS。ARM Cortex-M内核的嵌入式处理器，只能支持运行RTOS。

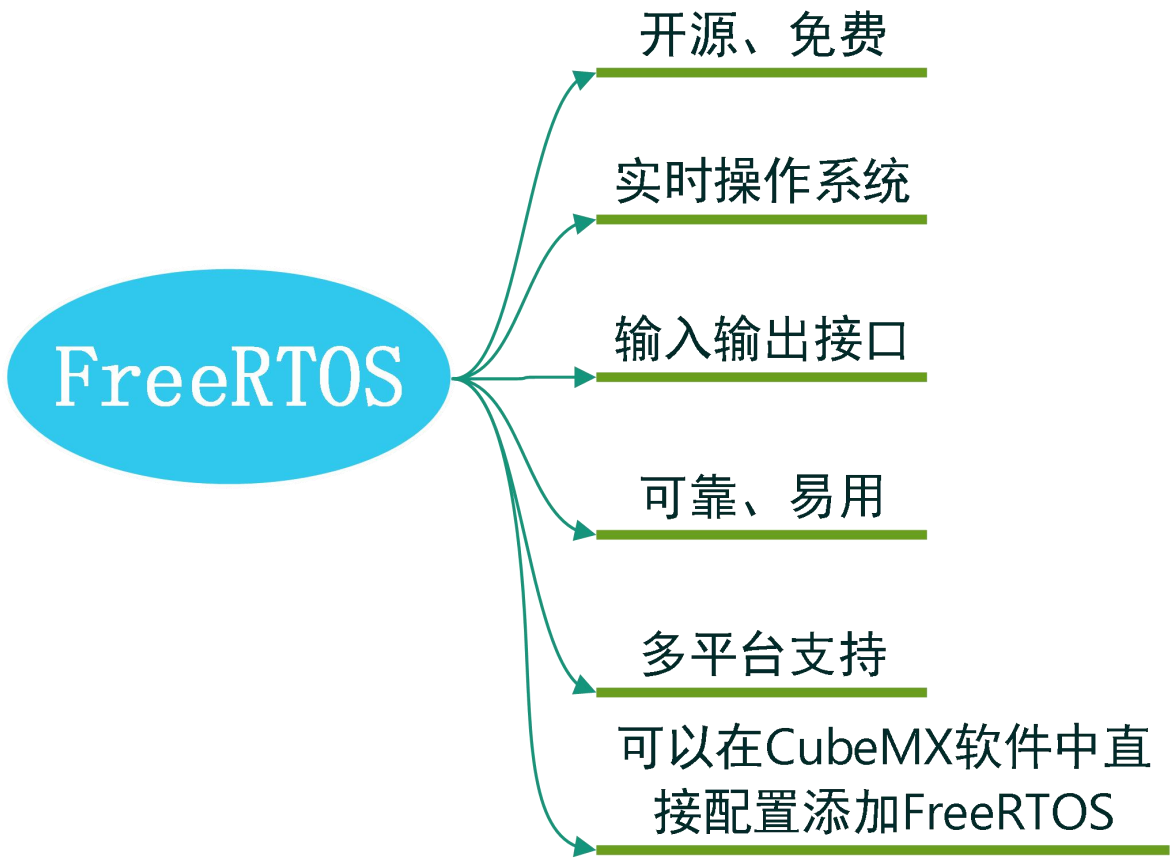
RTOS程序任务结构

如右图所示，在实时操作系统中，开发人员可以把要实现的功能划分为多个任务，每个任务负责实现其中的一部分，**每个任务都是**一个很简单的程序，通常是**一个死循环**。



如上图所示，在实时操作系统中，开发人员可以把要实现的功能划分为多个任务，每个任务负责实现其中的一部分，每个任务都是一个很简单的程序，通常是一个死循环。

FreeRTOS系统简介



现如今，业内流行的嵌入式RTOS：

- μC/OS-II
- μC/OS-III
- FreeRTOS
- μClinux
- RT-Thread
- Huawei OpenHarmony等等。

FreeRTOS的最新版本源码和相应官方手册都可以从其官网www.freertos.org获得。

FreeRTOS系统移植

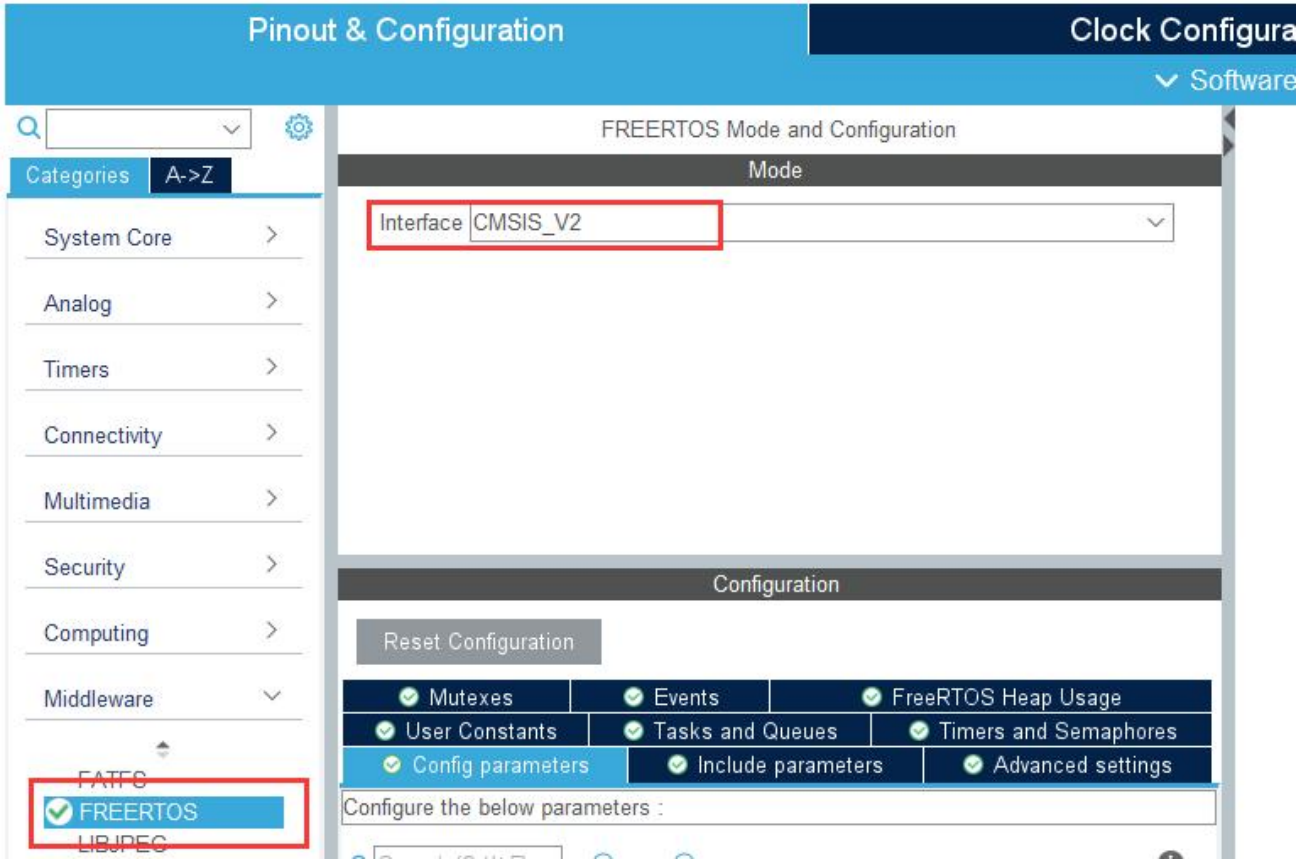
- CubeMX下的FreeRTOS是经过了ARM的CMSIS软件接口标准封装过的版本
- 将FreeRTOS添加到工程后，可以直接使用FreeRTOS原版的API函数
- 在MDK中还是推荐使用CMSIS封装好的API函数。

在CubeMX导出生成添加了FreeRTOS的MDK工程中，main函数内会自动添加osKernelInitialize和osKernelStart函数进行内核初始化和启动内核调度程序的操作，在两个函数之间还有用户任务的创建操作。

如何在CubeMX中添加FreeRTOS

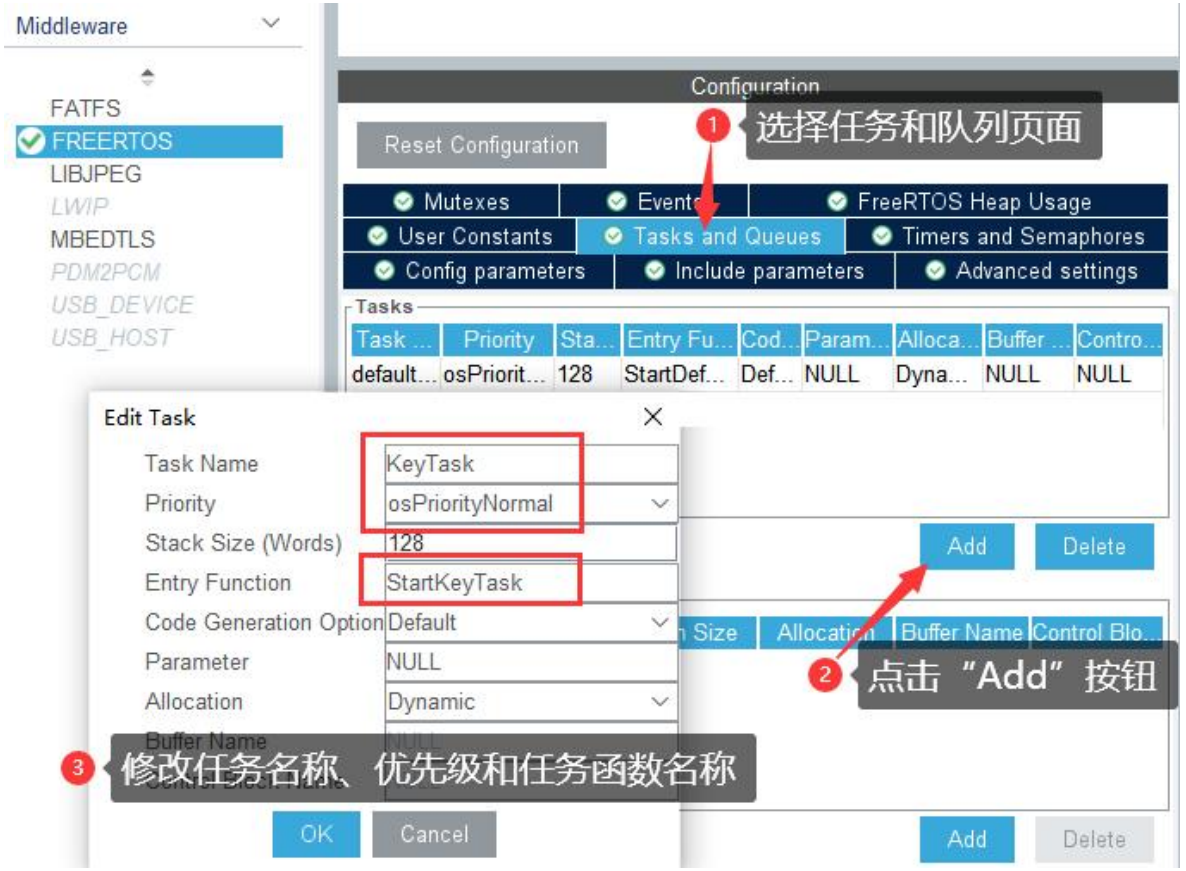
CubeMX中添加FreeRTOS的操作如下图所示，在“Pinout & Configuration”页面下，窗口左侧的Middleware中间件列表栏中选择添加。

注意，CubeMX下的FreeRTOS是经过了ARM的CMSIS软件接口标准封装过的版本，图中的接口版本有V1、V2两个版本可选，这两个版本有个别API函数不相同，本课程示例通常都推荐**选择CMSIS_V2**版本。



如何在CubeMX中添加FreeRTOS

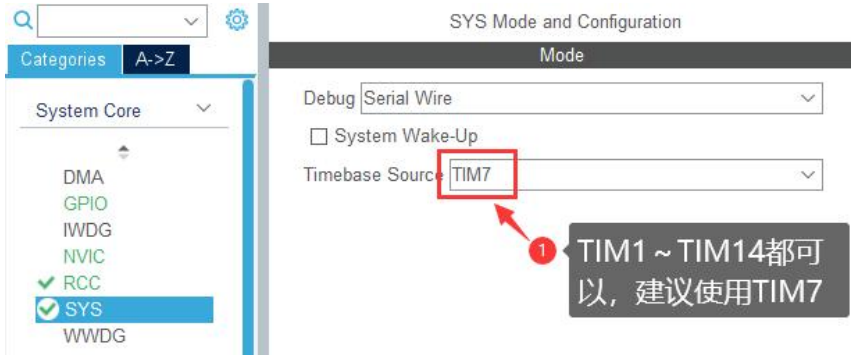
在CubeMX中添加FreeRTOS任务如下左图所示。最终导出的MDK工程源码就会自动生成该任务的任务函数和创建任务的代码，如下右图所示。

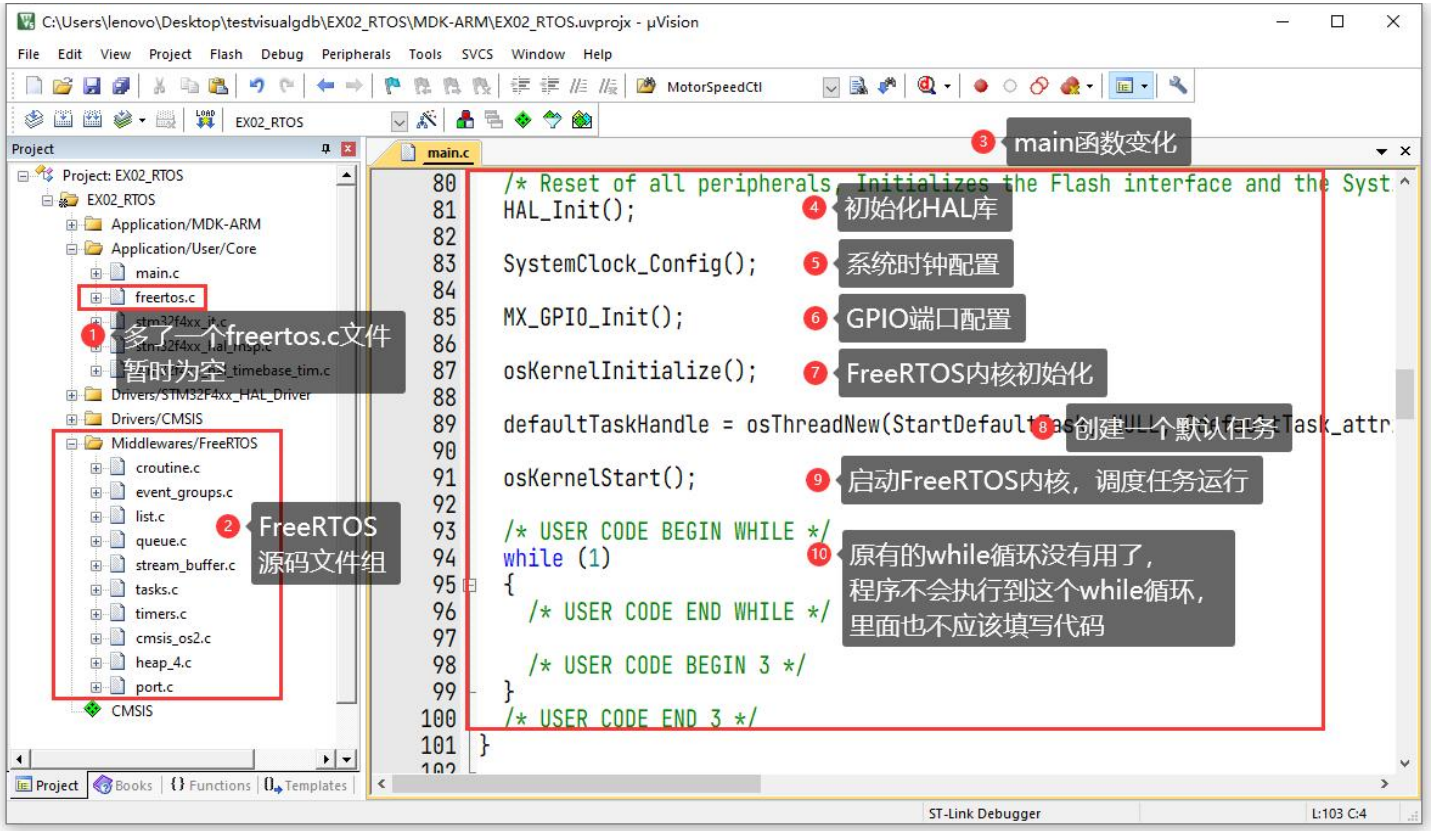



```
/* 定义KeyTask任务相关全局变量 */
osThreadId_t KeyTaskHandle;                          // KeyTask任务ID
const osThreadAttr_t KeyTask_attributes = {          // KeyTask任务属性
  .name = "KeyTask",                                 // 任务名称
  .stack_size = 128 * 4,                             // 任务栈空间大小
  .priority = (osPriority_t) osPriorityNormal,       // 任务优先级
};

void StartKeyTask(void *argument) { // KeyTask任务的入口函数
  for(;;)  {
    osDelay(1);
  }
}

/* 在main函数中创建任务 */
...
osKernelInitialize();    // 创建任务之前，需先初始化FreeRTOS内核
defaultTaskHandle = osThreadNew(StartDefaultTask,
                                NULL, &defaultTask_attributes);
KeyTaskHandle = osThreadNew(StartKeyTask, NULL, &KeyTask_attributes);
osKernelStart();         // 创建任务之后就可以启动内核任务调度了
```

如何在CubeMX中添加FreeRTOS

FreeRTOS的所有参数都暂时保持默认不变，最后导出工程时有提示建议修改SYS模块中的基准时钟源。如下左图所示，**修改为TIM7**。最后导出MDK工程，打开MDK工程后查看系统文件列表，如下右图所示，工程中多了一个FreeRTOS源码的文件组。





FreeRTOS内核控制相关函数

FreeRTOS内核控制的原版函数和CMSIS封装版本API函数对应关系如下所示：

功能分类	FreeRTOS原版函数	CMSIS v2封装函数	说明
内核信息和控制	无	osKernelInitialize	初始化RTOS内核
	无	osKernelGetInfo	获取RTOS内核信息
	xTaskGetSchedulerState	osKernelGetState	获取当前的RTOS内核状态
	vTaskStartScheduler	osKernelStart	启动RTOS内核调度程序
	xTaskGetTickCount xTaskGetTickCountFromISR	**osKernelGetTickCount**	**获取RTOS内核计数**
	无	osKernelGetTickFreq	获取RTOS内核节拍频率
	无	osKernelGetSysTimerCount	获取RTOS内核系统定时器计数
	无	osKernelGetSysTimerFreq	获取RTOS内核系统定时器频率
通用等待函数	vTaskDelay	**osDelay**	**等待超时（延时）**
	vTaskDelayUntil	osDelayUntil	等到指定的时间

FreeRTOS内核控制相关函数

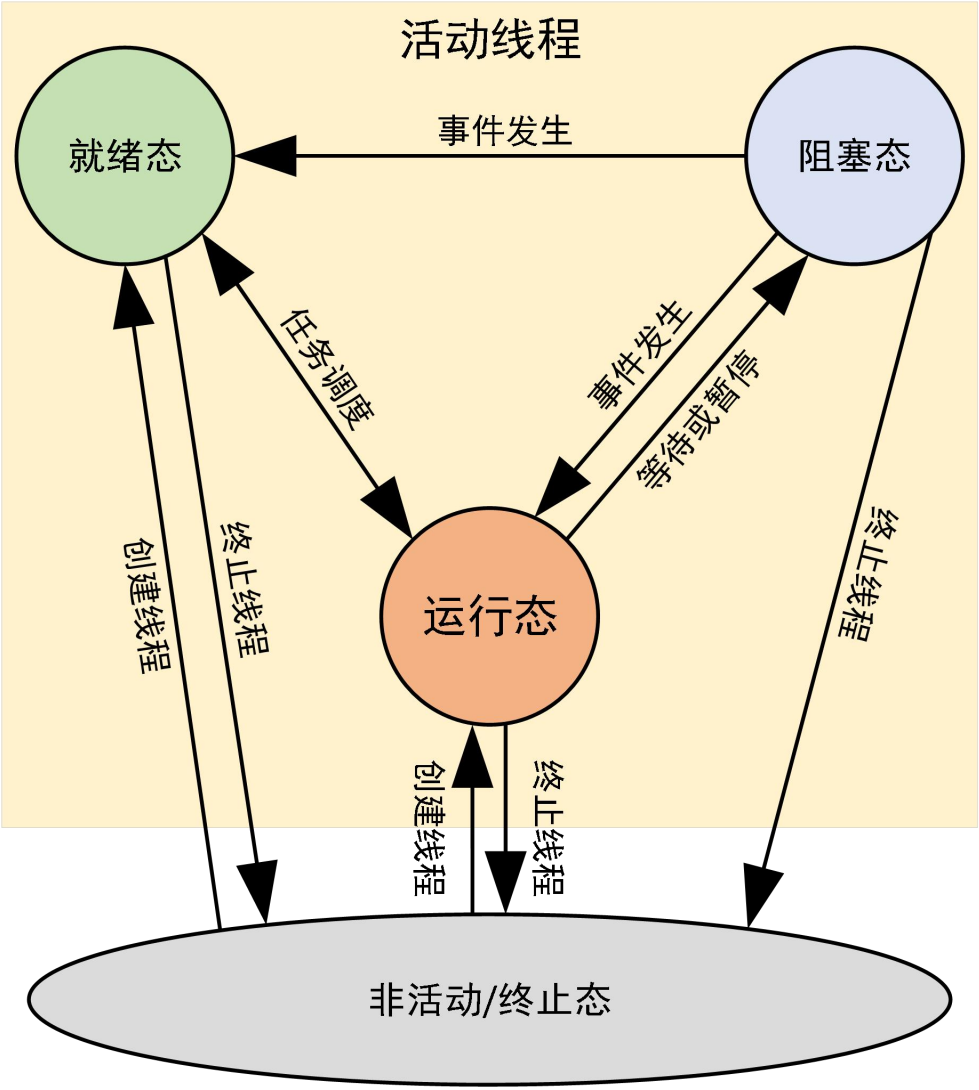
上一页的表中，**osDelay**函数是用户常用的毫秒级延时函数，**osKernelGetTickCount**是RTOS内核计数的获取函数，在系统内核节拍频率为默认的1000Hz时，这个内核计数即为**系统启动到当前时刻的累计时间毫秒数（简称系统时间戳）**。这两个函数是CMSIS多任务程序设计中常用的系统函数。

之前的裸机程序，在main函数的while循环中为了实现一个定时操作，使用了HAL_Delay延时函数来进行设计（参考实验1的LED秒闪程序代码）。而现在任务函数的for循环中，相比使用延时函数，课程更推荐使用**osKernelGetTickCount**函数来实现定时操作。类似功能代码如右图所示：

```
/* USER CODE END Header_StartDefaultTask */
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  uint32_t cur_tick, old_tick = 0;
  for(;;)
  {
    cur_tick = osKernelGetTickCount();
    if (cur_tick >= old_tick)
    {
      HAL_GPIO_TogglePin(L1_GPIO_Port, L1_Pin);
      old_tick = cur_tick + 500;
    }
    osDelay(1);
  }
  /* USER CODE END 5 */
}
```

多任务系统基本概念

- 多任务系统根据功能的不同，把整个系统分割成一个个的独立的函数。
- 每个函数内部和裸机程序的main函数一样都有一个无限循环。
- FreeRTOS中的任务（Task）在CMSIS中又称为线程（Thread）
- 操作系统内核一次只允许一个任务处于运行态，当运行态的任务被阻塞、延时、等待事件或暂停时，该任务将转入阻塞态，具有高优先级的下一个就绪态任务将转为运行态。



多任务系统基本概念

任务函数通常是一个不返回的无限循环函数，声明函数时必须要有一个void*类型的参数，没有返回值。

FreeRTOS不允许任务自行结束——**任务函数绝不能有return**，如果一个任务不需要了，需要显式的调用**osThreadExit**或者**osThreadTerminate**函数将其终止。

多任务系统基本概念

任务函数中有声明大数组或大量临时变量时，要注意创建任务时设置的栈空间大小。

任务栈空间太小可能会导致任务函数中定义的变量或者数组超出任务栈内存地址范围，导致程序跑飞等异常情况。

如果任务栈空间不够，那么需要修改stack_size的值，或者到CubeMX中修改任务栈大小（默认128×4字节），并重新导出工程。

修改任务栈空间大小时还要注意不能超过FreeRTOS的总堆栈大小，如果有超过，可以到CubeMX中修改FreeRTOS的TOTAL_HEAP_SIZE参数数值。对STM32F407系列的器件而言，TOTAL_HEAP_SIZE参数值建议不要超过65536。

多任务系统基本概念——任务优先级

任务优先级是RTOS内核给任务指定的优先等级，它决定了任务获取CPU资源的优先次序。**和STM32的中断优先级不同**，在FreeRTOS中的任务优先级**数值越小**，任务的**优先次序越低**，反过来，数值越大，优先次序越高。FreeRTOS允许创建多个相同优先级的任务，同优先级任务将使用时间片算法进行调度。FreeRTOS的优先级宏定义如右所示：

优先级宏定义	数值	备注
osPriorityNone	0	系统空闲任务使用
osPriorityIdle	1	用户常用优先级（注意不能超过configMAX_PRIORITIES宏定义数值）
osPriorityLow	8	
…		
osPriorityLow7	8+7	
osPriorityBelowNormal	16	
…		
osPriorityBelowNormal7	16+7	
osPriorityNormal	24	
…		
OsPriorityNormal7	24+7	
osPriorityAboveNormal	32	
…		
OsPriorityAboveNormal7	32+7	
osPriorityHigh	40	
…		
osPriorityHigh7	40+7	
osPriorityRealtime	48	
…		

多任务系统基本概念——任务优先级

根据不同任务的实时性要求程度，建议按如下规则设置任务优先级。

- 任务优先级
  - IRQ任务
    - 在中断中进行触发
      - 可以设置较高优先级
      - 可设为osPriorityHigh及其以上优先级
  - 高优先级后台任务
    - 串口消息处理
    - 按键检测等
      - 可设为osPriorityNormal及其附近的几个优先级
  - 低优先级的时间片调度任务
    - GUI界面显示
    - LED数码管显示
    - 不需要实时执行的任务
      - 可设为osPriorityLow及其附近的几个优先级

必须设置为阻塞式

只有这样，高优先级任务才会释放CPU资源，从而让低优先级任务有机会运行

FreeRTOS应用示例——多路闪灯控制

本次实验将演示一个多路闪灯控制示例应用，该示例使用FreeRTOS将按键扫描动作和闪灯控制分配在多个任务中独立运行，按下按键时使用全局变量通知闪灯任务控制对应LED灯。

参考教材4.4节内容，在CubeMX中选择器件STM32F407VE开始一个新的CubeMX工程。添加RCC晶振、参照下图添加按键和LED灯对应的GPIO、再添加FreeRTOS及其任务、配置系统时钟，最后设置工程名称为EX03，并导出MDK工程。注意为前6个LED添加一个闪灯任务，任务列表如下所示：

...	...	GPI...	GPIO mode	GPIO Pull-up/Pull-down	Maxi...	Use...
PE1	...	n/a	Input mode	Pull-up	n/a	K1
PE2	...	n/a	Input mode	Pull-up	n/a	K2
PE3	...	n/a	Input mode	Pull-up	n/a	K3
PE4	...	n/a	Input mode	Pull-up	n/a	K4
PE5	...	n/a	Input mode	Pull-down	n/a	K5
PE6	...	n/a	Input mode	Pull-down	n/a	K6
PE8	...	High	Output Push Pull	No pull-up and no pull-down	High	L1
PE9	...	High	Output Push Pull	No pull-up and no pull-down	High	L2
PE10	...	High	Output Push Pull	No pull-up and no pull-down	High	L3
PE11	...	High	Output Push Pull	No pull-up and no pull-down	High	L4
PE12	...	High	Output Push Pull	No pull-up and no pull-down	High	L5
PE13	...	High	Output Push Pull	No pull-up and no pull-down	High	L6
PE14	...	High	Output Push Pull	No pull-up and no pull-down	High	L7
PE15		High	Output Push Pull	No pull-up and no pull-down	High	L8

Mutexes | Events | FreeRTOS Heap Usage
User Constants | Tasks and Queues | Timers and Semaphores
Config parameters | Include parameters | Advanced settings

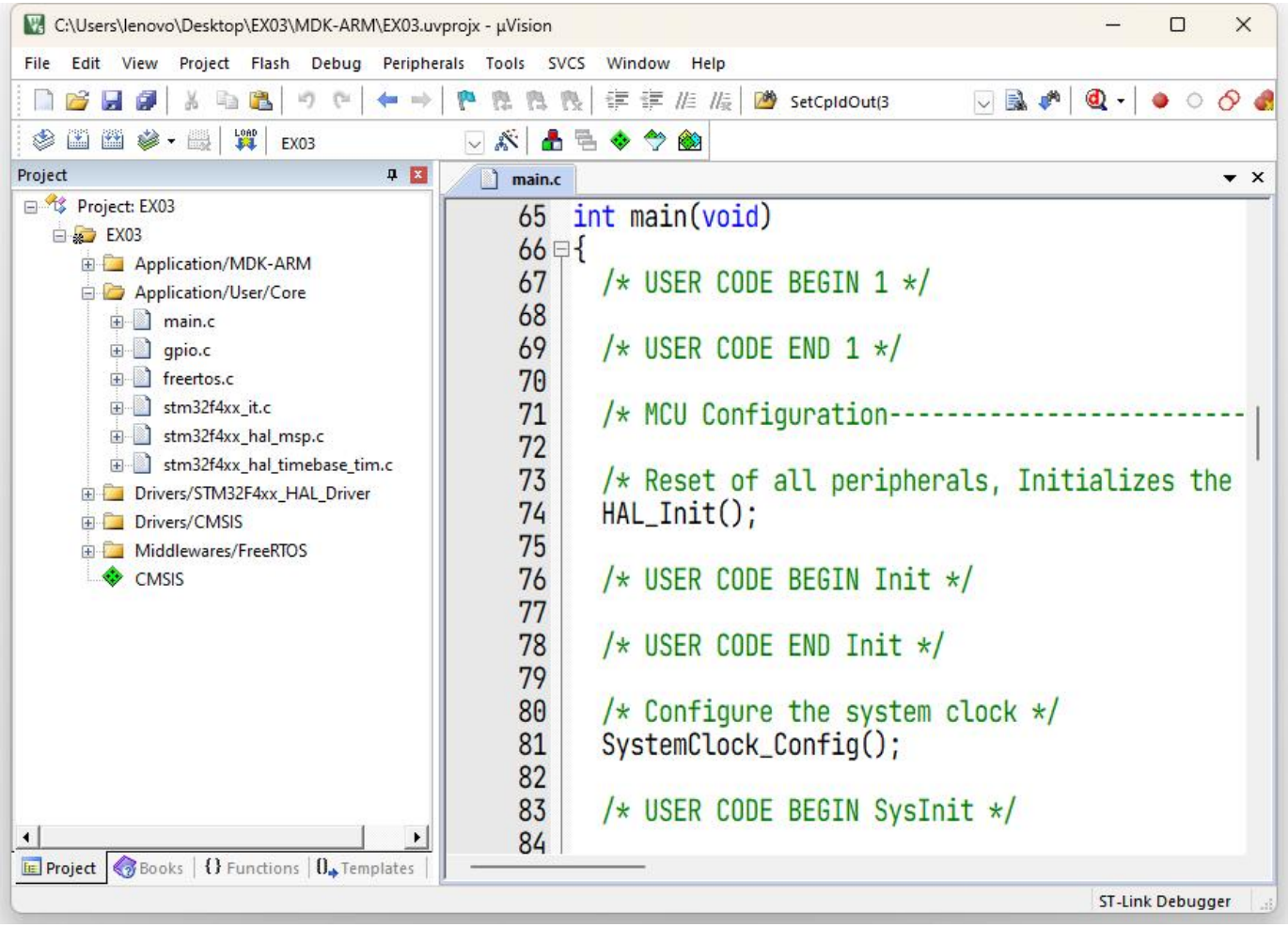
Tasks

Task Name	Priority	Stack ...	Entry Function	Code ...	Par...	Allocation	Buf...	Control B
defaultTask	osPriorityNormal	128	StartDefaultTask	Default	NULL	Dynamic	NULL	NULL
KeyTask	osPriorityNormal	128	StartKeyTask	Default	NULL	Dynamic	NULL	NULL
L1Task	osPriorityLow	128	StartTaskL1	Default	NULL	Dynamic	NULL	NULL
L2Task	osPriorityLow	128	StartTaskL2	Default	NULL	Dynamic	NULL	NULL
L3Task	osPriorityLow	128	StartTaskL3	Default	NULL	Dynamic	NULL	NULL
L4Task	osPriorityLow	128	StartTaskL4	Default	NULL	Dynamic	NULL	NULL
L5Task	osPriorityLow	128	StartTaskL5	Default	NULL	Dynamic	NULL	NULL
L6Task	osPriorityLow	128	StartTaskL6	Default	NULL	Dynamic	NULL	NULL

FreeRTOS应用示例——导出MDK工程

使用KEIL MDK软件打开EX03工程如右图所示：

可以看到，生成的MDK工程除了包含STM32F4的启动代码、HAL驱动、CMSIS库，主要提供了main.c、gpio.c、stm32f4xx_it.c和stm32f4xx_hal_msp.c这几个用户源程序文件。



FreeRTOS应用示例——main主函数结构

观察**main.c**程序代码，可以看到自动生成的main主函数结构如下：

```
int main(void){
    HAL_Init();                    // 初始化HAL库
    SystemClock_Config();          // 初始化系统时钟
    MX_GPIO_Init();                // 初始化GPIO端口
    osKernelInitialize();          // 初始化FreeRTOS内核
    MX_FREERTOS_Init();            // 初始化FreeRTOS任务、队列
    osKernelStart();               // 开始内核任务调度
    // 原有的while循环不再执行了，上行代码一直停留在内核调度循环中执行
    while (1) {
        // 此处添加代码无效
    }
}
```

FreeRTOS应用示例——编写LED和按键驱动

打开左侧工程文件列表中的**gpio.c**源文件，添加LED亮灯函数和按键扫描函数：

```c
/* USER CODE BEGIN 0 */
#include "cmsis_os.h"
/* USER CODE END 0 */



/* USER CODE BEGIN 2 */
// 设置单个LED灯亮灭状态翻转
void ToggleLED(uint8_t idx) {
    switch (idx){
        case 1:   HAL_GPIO_TogglePin(L1_GPIO_Port, L1_Pin);  break;
        case 2:   HAL_GPIO_TogglePin(L2_GPIO_Port, L2_Pin);  break;
        case 3:   HAL_GPIO_TogglePin(L3_GPIO_Port, L3_Pin);  break;
        case 4:   HAL_GPIO_TogglePin(L4_GPIO_Port, L4_Pin);  break;
        case 5:   HAL_GPIO_TogglePin(L5_GPIO_Port, L5_Pin);  break;
        case 6:   HAL_GPIO_TogglePin(L6_GPIO_Port, L6_Pin);  break;
        case 7:   HAL_GPIO_TogglePin(L7_GPIO_Port, L7_Pin);  break;
        case 8:   HAL_GPIO_TogglePin(L8_GPIO_Port, L8_Pin);  break;
        default:  break;
    }
}
```

```c
// 按键扫描函数，返回6个按键组合状态
uint16_t g_press_keysta = 0;                    // 全局变量，按键按住状态
uint16_t ScanKey(void) {
    // 按键按下时，K1 ~ K4低电平有效，K5、K6高电平有效
    uint8_t key = 0;
    if (HAL_GPIO_ReadPin(K1_GPIO_Port, K1_Pin) == GPIO_PIN_RESET) key |= K1_Pin;
    if (HAL_GPIO_ReadPin(K2_GPIO_Port, K2_Pin) == GPIO_PIN_RESET) key |= K2_Pin;
    if (HAL_GPIO_ReadPin(K3_GPIO_Port, K3_Pin) == GPIO_PIN_RESET) key |= K3_Pin;
    if (HAL_GPIO_ReadPin(K4_GPIO_Port, K4_Pin) == GPIO_PIN_RESET) key |= K4_Pin;
    if (HAL_GPIO_ReadPin(K5_GPIO_Port, K5_Pin) == GPIO_PIN_SET)   key |= K5_Pin;
    if (HAL_GPIO_ReadPin(K6_GPIO_Port, K6_Pin) == GPIO_PIN_SET)   key |= K6_Pin;
    g_press_keysta = key;
    if (key > 0){
        osDelay(10);                // 按键延时10毫秒消抖
        uint8_t key2 = 0;
        if (HAL_GPIO_ReadPin(K1_GPIO_Port, K1_Pin) == GPIO_PIN_RESET) key2 |= K1_Pin;
        if (HAL_GPIO_ReadPin(K2_GPIO_Port, K2_Pin) == GPIO_PIN_RESET) key2 |= K2_Pin;
        if (HAL_GPIO_ReadPin(K3_GPIO_Port, K3_Pin) == GPIO_PIN_RESET) key2 |= K3_Pin;
        if (HAL_GPIO_ReadPin(K4_GPIO_Port, K4_Pin) == GPIO_PIN_RESET) key2 |= K4_Pin;
        if (HAL_GPIO_ReadPin(K5_GPIO_Port, K5_Pin) == GPIO_PIN_SET) key2 |= K5_Pin;
        if (HAL_GPIO_ReadPin(K6_GPIO_Port, K6_Pin) == GPIO_PIN_SET) key2 |= K6_Pin;
        if (key == key2)    return key;         // 返回有效按键
        else        return 0;
    }
    return 0;
}
/* USER CODE END 2 */
```

FreeRTOS应用示例——添加驱动头文件

在**gpio.h**中添加LED和按键驱动的函数声明，以及流水灯状态变量的外部声明：

```
/* USER CODE BEGIN Prototypes */
// 设置单个LED灯亮灭状态翻转
void ToggleLED(uint8_t idx);
// 按键扫描函数，返回6个按键组合状态
uint16_t ScanKey(void);
extern uint16_t g_press_keysta;   // 按键按住状态
/* USER CODE END Prototypes */
```

准备添加流水灯控制的逻辑功能代码，打开MDK左侧工程文件列表中的**freertos.c**文件，在文件开头添加包含gpio.h头文件：

```
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "gpio.h"
/* USER CODE END Includes */
```

FreeRTOS应用示例——修改默认任务和按键扫描任务功能

在**freertos.c**中修改StartDefaultTask任务函数如下：

```
void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  for(;;) {
    osDelay(1);        // 默认任务空置，以备后用
  }
  /* USER CODE END StartDefaultTask */
}
void StartKeyTask(void *argument) {
  /* USER CODE BEGIN StartKeyTask */
  for(;;)
  {
    if (ScanKey() == 0)            // 扫描按键，如果没有按键动作
      HAL_GPIO_WritePin(L1_GPIO_Port, L1_Pin | L2_Pin | L3_Pin | L4_Pin | L5_Pin | L6_Pin, 1);
    osDelay(1);
  }
  /* USER CODE END StartKeyTask */
}
```

FreeRTOS应用示例——编写闪灯任务逻辑功能

在**freertos.c**中修改StartTaskL1任务函数代码如下所示：

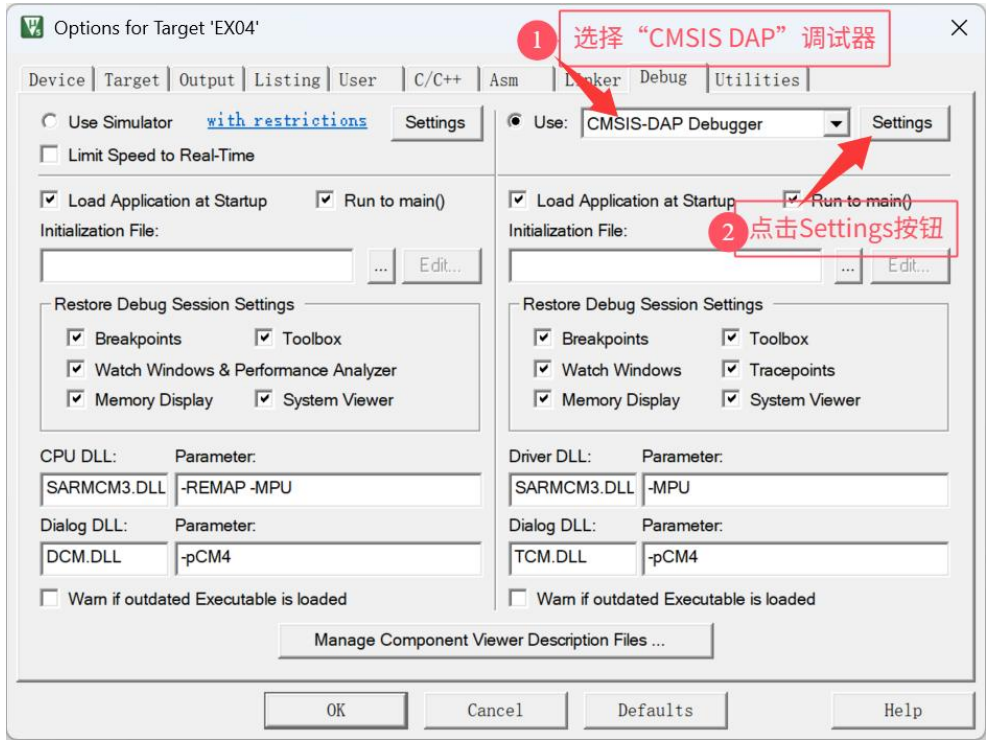
```
void StartTaskL1(void *argument)
{
  /* USER CODE BEGIN StartTaskL1 */
  /* Infinite loop */
  for(;;)
  {
    if (g_press_keysta & K1_Pin)  // 如果K1按住
    {
            ToggleLED(1);              // L1闪灯
            osDelay(100 + K1_Pin * 10);      // 延时一定时间
    }
    osDelay(1);
  }
  /* USER CODE END StartTaskL1 */
}
```

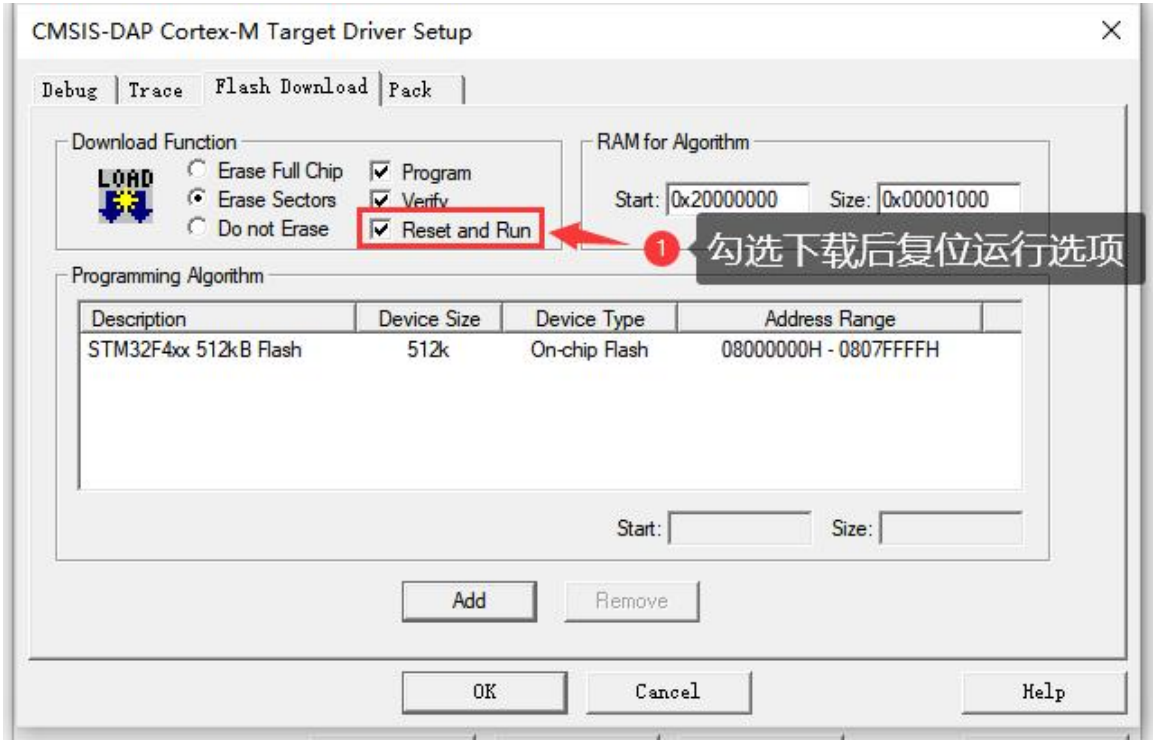
剩余5个闪灯任务代码同理，修改上述红色文字内容即可。

FreeRTOS应用示例——编译、下载、测试

添加代码完成后，连接学习板，参考下图进行MDK工程设置，确认调试器连接正常后，回到MDK主界面，按键盘F7快捷键编译工程。

编译成功后单击工具条上的“Download”按钮（快捷键F8）下载程序，按下学习板上的任意按键，就可以看到学习板上的闪灯效果，观察按键功能是否正常。





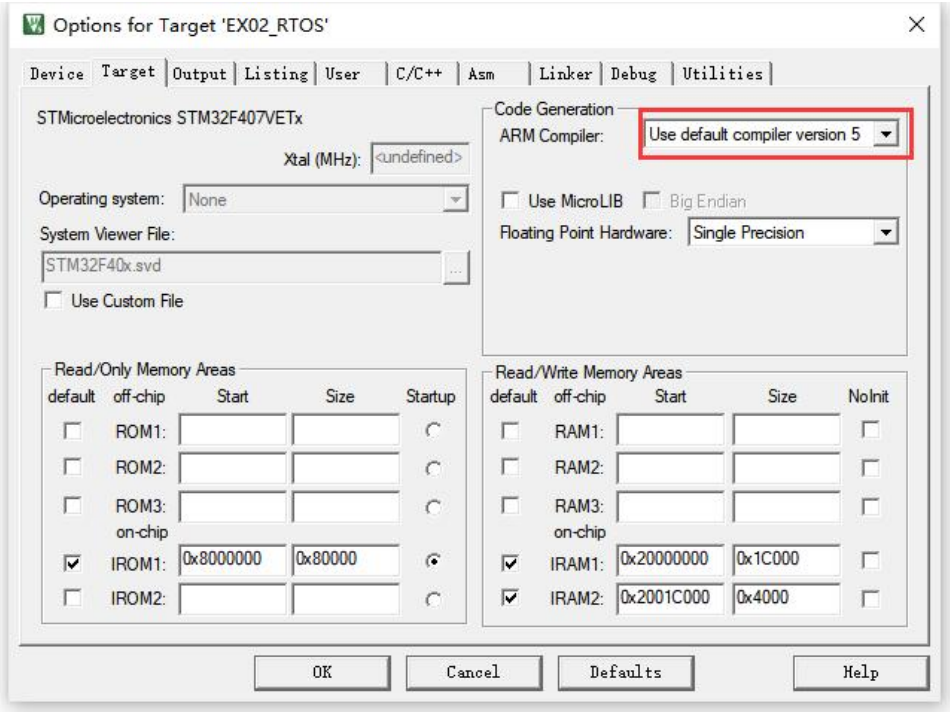
FreeRTOS应用编译器选择（MDK5.38以后版本）

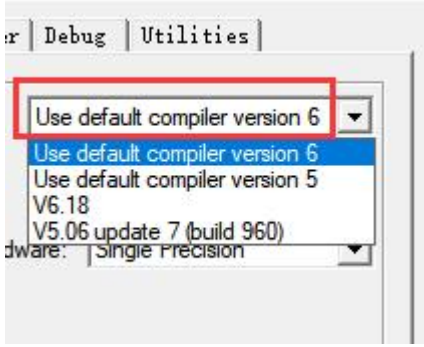
如下所示，KEIL MDK5.36之前的版本，工程默认的编译器是V5版本编译器。

这个版本的编译器效率比较低，工程包含文件较多时，第一次编译耗时很长。为了提高编译效率，右图的编译器列表中可以选择V6版本编译进行编译，编译速度将大大提高。

然而CubeMX导出工程代码时，复制的FreeRTOS源码中，有两个移植和V6版本编译器不兼容，导致最后选择V6版本编译器最后编译报错。

因此使用V6编译器时还需要修改FreeRTOS源码中的两个文件。





FreeRTOS源码文件替换（MDK5.38以后版本）

如下所示，打开CubeMX软件安装固件包的目录（一般在用户文件夹下的STM32Cube子目录中）。找到图中的两个**GCC版本**移植文件，**复制替换**工程文件夹中的**RVDS版本**移植文件。

复制替换后，在MDK中设置工程选项，选择V6版本编译器。然后点击工具条上的“Rebuild”按钮重新编译工程即可。

要注意的是，每次用CubeMX导出工程后，这两个文件都会恢复为RVDS版本。更直接的方法是替换固件包中FreeRTOS源码目录下的RVDS版本移植文件。不过这个做法会导致以后所有使用FreeRTOS的工程都只能用V6版本编译器进行编译。

实验内容

1. 示例工程实践练习

参照本次实验讲义内容，创建并复现EX03示例工程，最后在学习板上测试通过。

2. 任务应用练习

修改FreeRTOS.c文件中的默认任务和闪灯任务函数代码，把所有闪灯任务的闪灯逻辑都合并到默认任务中，然后把StartTaskL1 ~ StartTaskL6闪灯任务中的闪灯动作都注释掉。编译程序、下载测试闪灯效果。观察多个按键按住时是否闪灯异常，如果有，分析并说明原因。

3. 到学习通上的对应课程作业中，完成提交本次实验的**课堂作业。**

思考题：最后实验内容2基础上，再次修改默认任务代码，用本章第11页内容提到的时间戳方法重新设计闪灯延时，以解决多路闪灯异常问题。

下一次实验04

RTOS任务管理实验

请提前做好预习